Medical Computer Vision Lab

Cloud Platforms (AWS, Azure, GCP) and AI/ML Model Deployment Infrastructure:

Lab Standard

Maintainer: Ashery Mbilinyi

Version 1.0 : October 16, 2026

Contents

1 Purpose **3**

2 What "Cloud Platforms" Provide for ML **3**
2.1 Compute . . . 3
2.2 Storage . . . 3
2.3 Networking . . . 3
2.4 Identity & Access Management (IAM) . . . 3

3 What "Model Deployment Infrastructure" Means **3**

4 Equivalent Services: AWS vs Azure vs GCP **4**
4.1 Core Infrastructure Mapping (Quick Reference) . . . 4

5 Three Deployment Patterns (Most Common) **4**
5.1 Pattern A: Managed ML Endpoints (Fastest to Production) . . . 4
5.2 Pattern B: Kubernetes + Containers (Most Flexible) . . . 4
5.3 Pattern C: Serverless Containers (Great Middle Ground) . . . 5

6 Lab Standard: Recommended Default Choices **5**
6.1 Default Recommendations . . . 5
6.2 GPU Inference Guidance . . . 5

7 Deployment Building Blocks (What Every Cloud Deployment Needs) **5**
7.1 (1) Model Artifacts and Metadata . . . 5
7.2 (2) Containerized Inference Service . . . 6
7.3 (3) Traffic Management . . . 6
7.4 (4) Observability . . . 6
7.5 (5) Security . . . 6

8 Coding Examples **6**
8.1 Minimal FastAPI Inference App (Cloud-Friendly) . . . 6
8.2 Dockerfile (Inference Service) . . . 7
8.3 Local Run . . . 7
8.4 Example: Deploy to GCP Cloud Run (Conceptual) . . . 8
8.5 Example: Deploy to AWS (Conceptual ECS/Fargate) . . . 8
8.6 Example: Deploy to Azure (Conceptual Container Apps) . . . 8

9 CI/CD for Cloud Deployments (Minimal Standard) 8
9.1 Minimal GitHub Actions Sketch (Build Only) 8

10 Checklist (Code Review) 9

1 Purpose

This document explains cloud platforms (AWS, Azure, GCP) in the context of AI/ML workloads, and defines practical lab standards for **model deployment infrastructure** in the cloud. The goal is to ensure our cloud deployments are:

- **Reproducible**: consistent environments and versioned model artifacts
- **Secure**: least-privilege access and safe secret handling
- **Scalable**: supports increased traffic, batching, and multi-replica serving
- **Observable**: logs, metrics, and basic monitoring are always present
- **Rollback-ready**: model versions can be promoted and reverted safely

2 What "Cloud Platforms" Provide for ML

Cloud platforms provide standardized building blocks:

2.1 Compute

Where code runs:

- CPU VMs for web services and light inference
- GPU VMs for training and GPU inference
- managed container compute (serverless containers)

2.2 Storage

Where data and artifacts live:

- datasets, checkpoints, logs
- model artifacts (e.g., `.pt`, `.onnx`, tokenizers)

2.3 Networking

How services are exposed and connected:

- load balancers, DNS, HTTPS termination
- private networks (VPC/VNet), firewall rules

2.4 Identity & Access Management (IAM)

Who can access resources:

- service accounts/roles
- permission policies
- principle of least privilege

3 What "Model Deployment Infrastructure" Means

Model deployment infrastructure is the full system required to serve a model reliably:

- **Model packaging**: containerize inference + preprocessing code
- **Artifact storage**: store model files and metadata

- **Serving layer**: API endpoint (REST/gRPC)
- **Scaling**: autoscaling + load balancing + batching strategies
- **Monitoring**: latency, error rates, drift signals (when available)
- **Versioning**: model registry + rollout/rollback

4 Equivalent Services: AWS vs Azure vs GCP

4.1 Core Infrastructure Mapping (Quick Reference)

- **Compute VMs**: AWS EC2 / Azure VMs / GCP Compute Engine
- **Storage**: AWS S3 / Azure Blob Storage / GCP Cloud Storage
- **Container Registry**: AWS ECR / Azure ACR / GCP Artifact Registry
- **Kubernetes**: AWS EKS / Azure AKS / GCP GKE
- **Managed ML Platform**: AWS SageMaker / Azure ML / GCP Vertex AI
- **Secrets**: AWS Secrets Manager / Azure Key Vault / GCP Secret Manager

5 Three Deployment Patterns (Most Common)

5.1 Pattern A: Managed ML Endpoints (Fastest to Production)

Use cloud-managed model serving when you want simplicity and minimal DevOps.

Examples:

- AWS SageMaker Endpoints
- Azure ML Online Endpoints
- GCP Vertex AI Endpoints

Pros:

- fastest setup
- built-in scaling/monitoring integrations
- standardized deployment workflow

Cons:

- less control over runtime details
- cost can be higher for always-on endpoints

5.2 Pattern B: Kubernetes + Containers (Most Flexible)

Deploy a containerized inference API (FastAPI) on Kubernetes.

Pros:

- maximum control and portability
- strong scaling patterns and reliability
- standard infra across clouds

Cons:

- operational complexity
- requires cluster management knowledge

5.3 Pattern C: Serverless Containers (Great Middle Ground)

Run inference as a container on managed serverless container compute.
Examples:

- AWS: ECS Fargate (serverless containers)
- Azure: Container Apps
- GCP: Cloud Run

Pros:

- minimal ops
- easy autoscaling
- good for CPU inference and many API workloads

Cons:

- GPU inference support varies by platform/region and can be costly
- cold starts may matter for low-traffic services

6 Lab Standard: Recommended Default Choices

6.1 Default Recommendations

- **Internal lab demos**: Docker + FastAPI on a single VM (CPU/GPU as needed)
- **Simple production-style API**: serverless containers (Cloud Run / Container Apps / Fargate)
- **Research platform at scale**: Kubernetes (GKE/EKS/AKS) or Managed Endpoints

6.2 GPU Inference Guidance

Scaling GPU inference requires careful control:

- avoid too many workers competing for the same GPU memory
- prefer batching where latency allows
- consider queue-based designs for heavy workloads

7 Deployment Building Blocks (What Every Cloud Deployment Needs)

7.1 (1) Model Artifacts and Metadata

We store:

- model weights (`model.pt` / `model.onnx`)
- preprocessing config (normalization, transforms)
- label map / class definitions
- model card metadata (dataset, metrics, caveats)

7.2 (2) Containerized Inference Service

We serve inference via a container image:

- FastAPI app exposes `/health`, `/ready`, `/version`, `/predict`
- container pulls model artifact at startup or includes it in the image (decision depends on update frequency)

7.3 (3) Traffic Management

- HTTPS/TLS termination (managed load balancer or ingress)
- routing and versioning (v1/v2)
- optional canary rollouts

7.4 (4) Observability

Minimum:

- structured logs
- latency and error rate metrics

Recommended:

- tracing (distributed traces for microservices)
- drift monitoring when feasible

7.5 (5) Security

- IAM roles for services (least privilege)
- secrets stored in managed secret store (not in code or Docker images)
- private networking when possible

8 Coding Examples

8.1 Minimal FastAPI Inference App (Cloud-Friendly)

```python
# app.py
from __future__ import annotations

import os
from typing import List, Optional

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field

APP_VERSION = os.getenv("APP_VERSION", "dev")
MODEL_VERSION = os.getenv("MODEL_VERSION", "unknown")

app = FastAPI(title="MCV Cloud Inference API", version=APP_VERSION)

MODEL_LOADED = False

class PredictRequest(BaseModel):
    x: List[float] = Field(..., min_length=1)
    return_probs: bool = False
```

```python
class PredictResponse(BaseModel):
    y: int
    probs: Optional[List[float]] = None
    model_version: str

def load_model() -> None:
    # Replace with: download weights from S3/Blob/GCS or mount volume.
    global MODEL_LOADED
    MODEL_LOADED = True

def predict_impl(x: List[float]) -> tuple[int, List[float]]:
    score = sum(x)
    y = 1 if score > 0 else 0
    probs = [0.2, 0.8] if y == 1 else [0.8, 0.2]
    return y, probs

@app.on_event("startup")
def startup_event():
    load_model()

@app.get("/health")
def health():
    return {"status": "ok"}

@app.get("/ready")
def ready():
    if not MODEL_LOADED:
        raise HTTPException(status_code=503, detail="Model not loaded")
    return {"status": "ready"}

@app.get("/version")
def version():
    return {"app_version": APP_VERSION, "model_version": MODEL_VERSION}

@app.post("/predict", response_model=PredictResponse)
def predict(req: PredictRequest):
    if not MODEL_LOADED:
        raise HTTPException(status_code=503, detail="Model not loaded")

    y, probs = predict_impl(req.x)
    return PredictResponse(
        y=y,
        probs=probs if req.return_probs else None,
        model_version=MODEL_VERSION,
    )
```

8.2 Dockerfile (Inference Service)

```dockerfile
FROM python:3.10-slim

WORKDIR /app

RUN pip install --no-cache-dir fastapi uvicorn[standard] pydantic

COPY app.py /app/app.py

EXPOSE 8080
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8080"]
```

8.3 Local Run

```
docker build -t mcv-inference:latest .
docker run -p 8080:8080 \
  -e APP_VERSION=dev \
  -e MODEL_VERSION=v0 \
  mcv-inference:latest
```

8.4 Example: Deploy to GCP Cloud Run (Conceptual)

```
# Build and push to Artifact Registry (names simplified)
gcloud auth configure-docker
docker tag mcv-inference:latest REGION-docker.pkg.dev/PROJECT/REPO/mcv-inference
    :latest
docker push REGION-docker.pkg.dev/PROJECT/REPO/mcv-inference:latest

# Deploy
gcloud run deploy mcv-inference \
  --image REGION-docker.pkg.dev/PROJECT/REPO/mcv-inference:latest \
  --platform managed \
  --region REGION \
  --allow-unauthenticated
```

8.5 Example: Deploy to AWS (Conceptual ECS/Fargate)

```
# Build -> push to ECR -> create ECS service (high-level sketch)
# 1) Push image to ECR
# 2) Define task definition (CPU/mem/env vars)
# 3) Create service behind load balancer
# 4) Enable autoscaling rules
```

8.6 Example: Deploy to Azure (Conceptual Container Apps)

```
# Build -> push to ACR -> deploy to Azure Container Apps (high-level sketch)
# 1) Push image to ACR
# 2) Create Container App with ingress enabled
# 3) Configure autoscaling rules
```

9 CI/CD for Cloud Deployments (Minimal Standard)

A typical pipeline:

1. CI: lint + unit tests + inference smoke test
2. Build: Docker image
3. Push: image to registry (ECR/ACR/Artifact Registry)
4. Deploy: update service (Cloud Run / ECS / Container Apps / K8s)

9.1 Minimal GitHub Actions Sketch (Build Only)

```
name: Build

on:
  push:
    branches: [ "main" ]

jobs:
```

```
  docker-build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build
        run: docker build -t mcv-inference:latest .
```

10 Checklist (Code Review)

Before merging a cloud deployment project:

- Inference API implements `/health`, `/ready`, `/version`, `/predict`
- Dockerfile exists and builds successfully
- Model version is exposed (`/version`) and traceable to an artifact
- Secrets are not in the repo or image
- Logging exists and is structured enough to debug failures
- Scaling approach is documented (replicas/autoscaling/batching)